

# Point-Selection Fine-Tuning Framework for Robust Point Cloud Classification

Da Li<sup>1,2†</sup>, Chang Ma<sup>2†</sup>, and Dongfu Yin<sup>1\*</sup>

<sup>†</sup>Equal contribution. <sup>\*</sup>Corresponding author.

<sup>1</sup> Guangdong Laboratory of Artificial Intelligence and Digital Economy (Shenzhen),  
Shenzhen, China

<sup>2</sup> Shenzhen University, Shenzhen, China

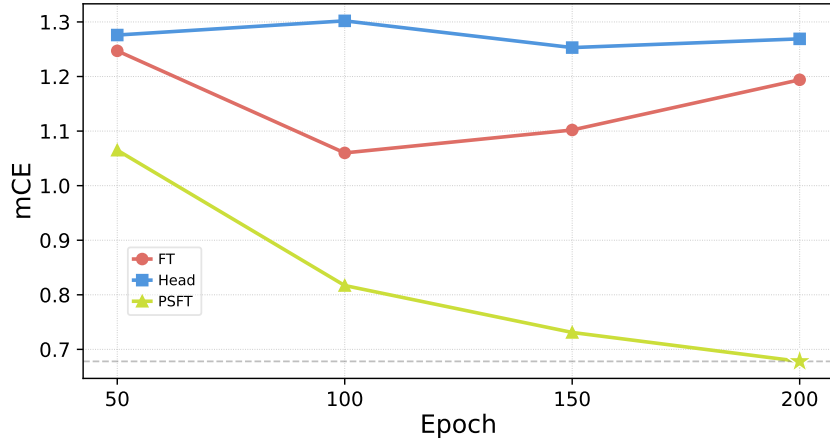
li944104439@gmail.com; 2510235035@mails.szu.edu.cn; yindongfu@gml.ac.cn

**Abstract.** Noisy and corrupted points can substantially degrade point cloud recognition performance, especially under challenging corruption settings. In particular, full fine-tuning of 3D pre-trained models may amplify the influence of outliers and overwrite robustness priors learned during pre-training, while naive parameter-efficient adaptation remains sensitive to corrupted tokens. To address this issue, we propose PSFT, a point-selection fine-tuning framework that improves robustness while remaining parameter-efficient. PSFT first estimates point-wise influence from pre-pooling features and adaptively retains minimally influential points to suppress outliers. Based on the selected subset, a prompt generation branch predicts layer-wise prompt tokens and injects them into a frozen backbone for lightweight downstream adaptation. To further mitigate residual noise after selection, we append a lightweight feature filter with bottleneck MLP transformation and Beta-gated residual blending to refine patch-token representations before prediction. Extensive experiments show that PSFT consistently reduces corruption error on ModelNet-C and ModelNet40-C across all tested 3D pre-trained backbones, while achieving the strongest ScanObjectNN-C results with ULIP-2 and Uni3D-B among the evaluated tuning strategies. Our implementation can be found at <https://github.com/CVChMA/PSFT/tree/master>.

**Keywords:** Point clouds · Robust classification · Parameter-efficient fine-tuning

## 1 Introduction

Point cloud understanding is fundamental to applications ranging from 3D object recognition to autonomous-driving perception, where LiDAR-camera fusion supports accurate 3D detection [2]. However, noisy and corrupted points can severely degrade model performance, especially under challenging corruption settings such as ModelNet40-C [23]. Recent studies have made substantial progress on robust point cloud modeling [40, 21, 20, 33], but robust adaptation of 3D pre-trained models under corruption remains insufficiently explored. Our



**Fig. 1. Effect of Fine-Tuning Strategy on Robustness.** mCE denotes mean corruption error. FT updates all parameters, Head tunes only the classifier, and PSFT is our method.

preliminary study on Point-MAE shows that full fine-tuning yields substantially higher corruption error than PSFT.

To analyze this behavior, we fine-tune pre-trained Point-MAE for 200 epochs and evaluate robustness every 50 epochs. We compare three strategies under the same schedule: full fine-tuning, head-only tuning, and PSFT. As shown in Figure 1, full fine-tuning improves robustness at early epochs but later degrades, while head-only tuning remains stable yet with relatively high mCE. In contrast, PSFT consistently maintains the lowest corruption error. These results suggest that full fine-tuning can overwrite robust pre-trained priors, whereas head-only tuning lacks sufficient adaptation capacity.

We further evaluate IDPT [36], a representative parameter-efficient fine-tuning (PEFT) baseline, on pre-trained Point-MAE and find that PEFT alone remains sensitive to corrupted inputs. This indicates that noisy points can still interfere with lightweight adaptation modules. Motivated by this observation, we adopt the point selection strategy of [10] to filter input points before feeding them into both the PEFT branch and the frozen pre-trained network. Since residual noise may still remain after selection, we introduce a Feature Filter Module (FFM) that refines token features with a lightweight MLP and a Beta-gated residual connection, balancing denoising and information preservation.

The novelty of PSFT lies in this robustness-oriented coupling: selection protects both prompt generation and frozen-backbone inference, while feature filtering addresses noise that survives input selection.

We validate our method on three corruption benchmarks: ScanObjectNN-C [26], ModelNet-C [21], and ModelNet40-C [23]. Our objective is to improve corruption robustness while maintaining adaptation efficiency. To this end, we design lightweight modules that preserve robust priors in frozen pre-trained backbones. In addition, we provide targeted analyses to disentangle module-wise con-

tributions and assess the transferability of robustness gains across benchmarks. The primary contributions of our work can be summarized as follows:

- We propose PSFT, a point-selection fine-tuning framework for robust 3D adaptation that combines point selection, prompt tuning on a frozen backbone, and feature filtering.
- PSFT achieves consistent robustness gains on ModelNet-C and ModelNet40-C across diverse 3D pre-trained backbones, and improves strong-backbone results on ScanObjectNN-C.
- We conduct systematic ablation and sensitivity analyses to quantify module-wise contributions and the impact of residual blending design choices on robustness.

## 2 Related Work

### 2.1 Point Cloud Classification

Point cloud classification methods can be grouped into voxel-based, multi-view, and point-based paradigms. Voxel-based approaches discretize geometry into regular grids [5], while multi-view methods project 3D shapes into 2D renderings for image-style recognition [6]. Point-based methods process unordered points directly and avoid discretization artifacts, making them the dominant direction for fine-grained 3D recognition.

Within point-based modeling, the architecture trajectory has moved from shared-MLP designs to stronger local aggregation. PointNet [16] establishes permutation-invariant feature extraction, PointNet++ [17] introduces hierarchical neighborhoods, and DGCNN [28] improves relational modeling with dynamic graphs. Subsequent models such as PCT [7], PointMLP [14], and PointNeXt [19] further improve accuracy-efficiency trade-offs on standard classification benchmarks.

Recent progress is increasingly driven by pre-training and multimodal alignment. Transformer backbones improve long-range interaction modeling [41, 34]. Masked or reconstruction-based pre-training further improves transferability [15, 18]. Large-scale multimodal frameworks also strengthen 3D representation quality [3, 31, 32, 42]. As backbones become stronger, adaptation under corrupted inputs becomes a central practical challenge.

### 2.2 Parameter-Efficient Fine-Tuning in 3D Domain

Parameter-efficient fine-tuning (PEFT) updates a small subset of parameters while keeping most backbone weights frozen. In the vision domain, representative paradigms include prompt tuning and adapter tuning, as exemplified by VPT [8] and AdaptFormer [4]. Their main advantage is reduced optimization cost with competitive transfer performance.

In 3D point cloud learning, PEFT has developed rapidly in recent years. IDPT [36] introduces instance-aware prompts for point cloud adaptation, Point-PEFT [24] combines point-prior prompts with geometry-aware adaptation, and PPT [22] explores efficient prompt-adapter integration. DAPT [43] provides a unified prompt-adapter design. More recent methods, such as GAPrompt [1] and vision-semantic prompting [37], further incorporate geometric priors or external semantic guidance into frozen 3D backbones.

Despite strong clean-data transfer, most PEFT studies primarily optimize accuracy-efficiency trade-offs and report limited robustness analysis under corruption. Prior work also indicates that fine-tuning choices can induce nontrivial robustness-accuracy trade-offs [11]. Motivated by this gap, our work focuses on robust PEFT for corrupted point clouds, rather than clean-data PEFT alone.

### 2.3 Advances in 3D Point Cloud Model Robustness

Robust point cloud research includes architecture-level invariance, denoising and outlier suppression, and certified defense. Geometry-aware methods improve invariance to structural perturbations [40]. Denoising-oriented approaches explicitly mitigate noisy observations [20, 33]. In adversarial settings, PointGuard studies certified robustness guarantees for 3D classification [12].

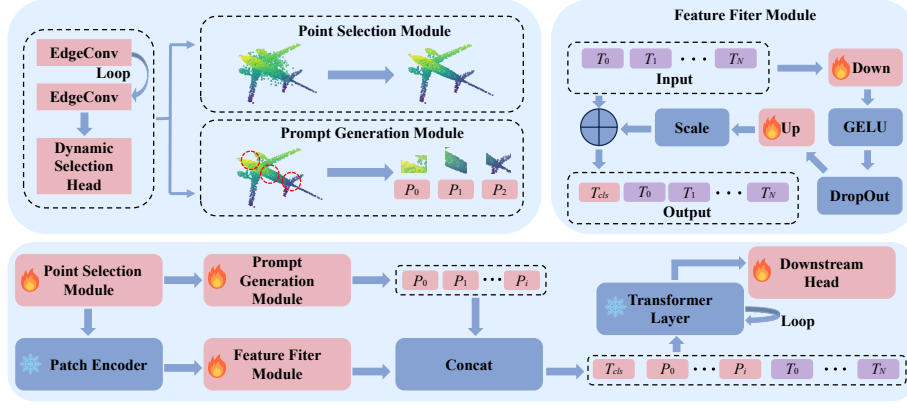
Unlike coordinate-space denoisers that reconstruct or resample point sets, PSFT couples influence-based selection with parameter-efficient adaptation and token-level refinement; the two directions are therefore complementary rather than interchangeable.

Another line investigates robustness from the representation perspective. Refocusing [10] shows that highly influential points under pooling are often corrupted outliers, and suppressing their dominance can improve robustness. Robustness under incomplete or partial observations is also studied by transformer-based inference on partial inputs [35], voting-based aggregation [38], and ensemble partial-point reasoning such as EPiC [9].

Robustness benchmarks provide standardized evaluation under diverse corruption settings. ModelNet-C [21] and ModelNet40-C [23] emphasize controlled synthetic corruptions, while ScanObjectNN-C [26] focuses on more realistic corruption patterns. These benchmarks consistently show that high clean-data transfer performance does not necessarily imply strong corruption robustness after adaptation. Our method is designed in this benchmark-driven setting, with emphasis on robust and parameter-efficient adaptation of frozen pre-trained models.

## 3 Methodology

We observe that full fine-tuning often provides limited robustness gains for 3D pre-trained models, and may even deteriorate performance under corruption. This suggests that the challenge is not only how to adapt the model to the downstream task, but also how to prevent corrupted points from destabilizing



**Fig. 2. Overview of PSFT.** The point selection module ranks points by contribution and retains a robust subset. The prompt generation module uses an EdgeConv-based branch to produce prompt tokens for a frozen transformer backbone. The feature filter module refines patch features by suppressing residual noise while preserving informative signals through residual blending.

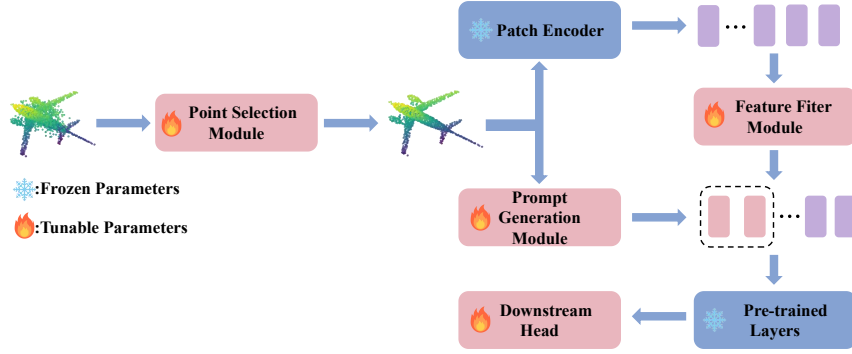
the adaptation process itself. Prior work has shown that outliers in corrupted point clouds can become highly influential under pooling and severely distort the resulting representation [10]. Motivated by this, we propose a point-selection fine-tuning framework, PSFT, which explicitly reduces the impact of outliers while preserving the robust representations of the frozen backbone. Figure 2 and Figure 3 illustrate the overall design.

Specifically, PSFT first applies a point selection module to retain minimally influential points and suppress outliers (Sec. 3.1). It then keeps the backbone frozen and introduces a prompt generation module for lightweight task adaptation while preserving the robustness of the pre-trained model (Sec. 3.2). Finally, a feature filter module refines token features after the frozen encoder before prediction (Sec. 3.3).

### 3.1 Point Selection Module

We adopt the refocusing strategy of [10] as our point selection module. It serves two purposes: it ranks points according to their contribution to the pooled representation, and it adaptively determines how many points should be retained for subsequent processing.

*Measuring Point-Wise Contribution via Max-Pooling.* Let  $X_f \in \mathbb{R}^{N \times \kappa}$  denote the final point-wise (or patch-wise) feature map before global pooling, where  $N$  is the number of input elements and  $\kappa$  is the channel dimension. For point-based backbones,  $X_f$  is extracted from the last point feature map before pooling; for transformer backbones, it is taken from the patch-token feature map before the final pooling. We quantify the contribution of each point by counting how often it provides the maximum activation across feature channels.



**Fig. 3. PSFT Workflow.** PSFT first selects minimally influential points to suppress outliers, then generates layer-wise prompt tokens from the selected subset, and finally refines encoder features with a lightweight filter module before prediction.

Formally, the contribution score  $I_F(j)$  for point  $j \in \{1, \dots, N\}$  is defined as:

$$I_F(j) = \sum_{k=1}^{\kappa} \mathbb{I} \left( j = \arg \max_{n \in \{1, \dots, N\}} X_f(n, k) \right), \quad (1)$$

where  $\mathbb{I}(\cdot)$  is the indicator function. For each feature channel  $k$ , the point providing the maximum activation receives one vote. Hence,  $I_F(j)$  reflects how strongly point  $j$  influences the final pooled representation.

*Adaptive Point Selection via Entropy-Guided Focus Estimation.* To determine the appropriate number of points to retain in a corrupted point cloud, the entropy of point-wise feature importance is computed and used to guide the adaptive point selection process.

Following the previously defined point contribution score  $I_F(j)$ , we obtain a contribution vector  $I_F = (I_F(1), \dots, I_F(N))$ . We normalize it into a probability distribution  $p$  over the point cloud:

$$p_j = \frac{I_F(j)}{\sum_{i=1}^N I_F(i) + \varepsilon}, \quad (2)$$

where  $\varepsilon$  is a small constant for numerical stability. We define  $0 \log 0 = 0$  when computing entropy.

We then compute the entropy of this distribution:

$$H(p) = - \sum_{j=1}^N p_j \ln(p_j), \quad (3)$$

and normalize it to eliminate the influence of varying point cloud sizes:

$$H_n(p) = \frac{H(p)}{\ln(N)}. \quad (4)$$

The normalized entropy  $H_n(p) \in [0, 1]$  measures how uniformly the network distributes its focus across the point cloud. A higher entropy indicates that the representation depends on many points, while a lower entropy suggests that only a small subset dominates the pooled feature.

To adaptively determine how many points should be retained, we define the selected point number  $S$  as:

$$S = \text{clip}(\text{round}((1 - H_n(p)) \cdot N), 1, N). \quad (5)$$

After computing  $S$ , we sort points in ascending order of  $I_F(j)$  and retain the first  $S$  minimally influential points. This strategy follows the observation of [10] that highly influential points are more likely to correspond to corrupted outliers. When the representation is highly concentrated (low entropy), we preserve more points to avoid over-trusting a very small subset; when the representation is more dispersed, a smaller robust subset is typically sufficient.

Ignoring rounding and clipping, the retained ratio is  $S/N = 1 - H_n(p)$ ; hence a change  $\Delta H_n$  changes the ratio by  $-\Delta H_n$ . This bounded monotone rule makes the selection strength adaptive rather than dataset-fixed.

### 3.2 Prompt Generation Module

To adapt the model while preserving the robustness of the pre-trained backbone, we adopt a prompt-based tuning strategy that inserts one learnable prompt token into each transformer layer. The prompts are conditioned on the selected point subset through an EdgeConv-based prompt generator [28], which captures local geometric structure and produces a compact global descriptor for prompt prediction.

Unlike full fine-tuning, PSFT keeps the backbone frozen rather than updating its parameters, since modifying the pre-trained model may compromise the robustness it has already learned. Instead, PSFT performs task adaptation through a prompt branch that predicts layer-specific prompt vectors from the selected points, while the frozen encoder branch continues to produce the patch tokens. This design introduces task-specific flexibility while minimizing interference with the robust representations of the backbone.

Formally, let  $T_{\text{cls}} \in \mathbb{R}^{1 \times d}$  denote the classification token, let  $T_i \in \mathbb{R}^{M \times d}$  denote the patch tokens entering the  $i$ -th transformer layer, and let  $P_i \in \mathbb{R}^{1 \times d}$  denote the prompt token predicted for that layer. The prompt-augmented token sequence processed by the  $i$ -th transformer layer  $L_i$  is

$$x_i = L_i([T_{\text{cls}}, P_i, T_i]). \quad (6)$$

This formulation inserts exactly one prompt token per layer and avoids cumulative prompt growth. By conditioning  $P_i$  on the selected points, PSFT adapts the model to the downstream task without modifying the frozen backbone, thereby better preserving robustness under corruption.

### 3.3 Feature Filter Module

Although the Point Selection Module filters out a significant portion of corrupted or less informative points, the retained subset may still contain residual noise due to sampling imperfections or perturbations such as jitter. To further improve representation quality, we introduce a Feature Filter Module (FFM) after the frozen encoder to refine patch-token features before prediction.

The Feature Filter Module is a lightweight funnel-shaped multi-layer perceptron (MLP) applied to patch tokens after the frozen encoder and before the final pooling stage. We keep the classification token unchanged and apply the filter token-wise to each patch token. For an input token  $x_o \in \mathbb{R}^d$ , the filtered token  $x_f \in \mathbb{R}^d$  is obtained as

$$x_f = \text{DropOut}(\text{GELU}(x_o \cdot W_{\text{down}})) \cdot W_{\text{up}}, \quad (7)$$

where  $W_{\text{down}} \in \mathbb{R}^{d \times d'}$  and  $W_{\text{up}} \in \mathbb{R}^{d' \times d}$  are projection matrices with a bottleneck dimension  $d' < d$ , and GELU and Dropout are used to introduce non-linearity and regularization.

While filtering suppresses noise, it may also weaken useful cues. To balance this trade-off, we blend the filtered and original features through a residual connection. During training, the blending ratio is sampled as  $\alpha \sim \text{Beta}(\lambda, \lambda)$ ; one  $\alpha$  is sampled per input sample and shared across its tokens.

$$x_{\text{final}} = (1 - \alpha) \cdot x_f + \alpha \cdot x_o. \quad (8)$$

The Beta distribution serves two purposes. First, it provides a bounded interpolation mechanism in  $[0, 1]$ , allowing smooth control over the contribution of the filtered and original features. Second, the stochasticity introduced during training acts as an implicit regularizer, improving robustness to corrupted inputs. In the analysis of Sec. 4, we examine how different training-time  $\lambda$  settings and test-time  $\alpha$  values affect this trade-off.

For  $\alpha \sim \text{Beta}(\lambda, \lambda)$ ,  $\mathbb{E}[\alpha] = 1/2$  and  $\text{Var}(\alpha) = 1/[4(2\lambda + 1)]$ , so smaller  $\lambda$  exposes training to a broader range of mixtures. Moreover,  $x_{\text{final}} - x_o = (1 - \alpha)(x_f - x_o)$ , explicitly bounding the correction applied to the frozen feature.

## 4 Experiments

### 4.1 Datasets and Basic Settings

*Datasets.* We evaluate our method on three representative corruption benchmarks: ModelNet-C, ModelNet40-C, and ScanObjectNN-C. These datasets cover both synthetic and real-world corruption settings, enabling a comprehensive assessment of robustness under different degradation conditions.

*Implementation details.* Models are trained on the clean versions of the corresponding datasets, such as ModelNet40 [29] and ScanObjectNN [25]. Evaluations are then conducted on their corruption test suites, namely ModelNet-C [21], ModelNet40-C [23], and ScanObjectNN-C [26]. We use the AdamW optimizer with a learning rate of  $5 \times 10^{-4}$  and train for 300 epochs. For ModelNet-C and ScanObjectNN-C, we follow the original benchmark protocols and report mean Corruption Error (mCE,  $\downarrow$ ). For ModelNet40-C, we report mean Error Rate (mER, %,  $\downarrow$ ), i.e., the average classification error over all corruption types in the benchmark. Unless otherwise specified, the main paper reports results without data augmentation so that the effect of robust adaptation can be isolated more clearly; results with WOLFMix [21] augmentation are deferred to the supplementary material.

## 4.2 Comparison Results

*Overall Performance.* We evaluate PSFT on three corrupted point cloud benchmarks: ModelNet-C, ModelNet40-C, and ScanObjectNN-C. As shown in Table 1, Table 2, and Table 3, PSFT improves robustness on all four evaluated backbones on ModelNet-C and ModelNet40-C, and on the stronger ULIP-2 and Uni3D-B backbones on ScanObjectNN-C. The evaluated pre-trained backbones include PointBERT [34], PointMAE [15], Uni3D [42], and ULIP-2 [31, 32].

*Results on ModelNet-C and ModelNet40-C* Table 1 shows that PSFT consistently reduces mCE for all four evaluated pre-trained backbones on ModelNet-C. The best result among the compared methods is achieved by ULIP-2 with PSFT, which reaches an mCE of 0.530. Table 2 shows a similar trend on the broader ModelNet40-C suite: PSFT substantially lowers mER for strong backbones such as ULIP-2 and Uni3D-B, indicating that the proposed tuning strategy generalizes across both synthetic corruption benchmarks. The corresponding results with augmentation are reported in the supplementary material.

*Results on ScanObjectNN-C.* On the more challenging real-world benchmark ScanObjectNN-C, PSFT achieves the best mCE among the compared PEFT variants, reaching 0.685 on ULIP-2. The improvements obtained with ULIP-2 and Uni3D-B support the robustness benefit of point selection and feature filtering on strong backbones, while the PointBERT and PointMAE results indicate that the gains do not transfer uniformly to every backbone in real-world settings.

Relative to backbone-specific full fine-tuning, the ScanObjectNN-C mCE changes are +0.039, +0.119, -0.092, and -0.099 for Point-BERT, Point-MAE, ULIP-2, and Uni3D-B, respectively. This mixed pattern contrasts with the uniform synthetic-benchmark gains and indicates backbone-dependent transfer under real corruptions.

## 4.3 Further Analysis

*Impact of Each Component.* We conduct a series of controlled experiments to investigate the impact of each component of PSFT, as summarized in Table 5. We

**Table 1. Experimental Results on ModelNet-C.** OA and mCE denote overall accuracy and mean corruption error, respectively.

| Model           | OA $\uparrow$ | mCE $\downarrow$ | Scale        | Jitter       | Drop-G       | Drop-L       | Add-G        | Add-L        | Rotate       |
|-----------------|---------------|------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| PointNet [16]   | 90.7%         | 1.422            | 1.266        | 0.642        | 0.500        | 1.072        | 2.980        | 1.593        | 1.902        |
| RSCNN [13]      | 92.3%         | 1.130            | 1.074        | 1.171        | 0.806        | 1.517        | 0.712        | 1.153        | 1.479        |
| PAConv [30]     | 93.6%         | 1.104            | 0.904        | 1.465        | 1.000        | 1.005        | 1.085        | 1.298        | 0.967        |
| PointNet++ [17] | 93.0%         | 1.072            | 0.872        | 1.177        | 0.641        | 1.802        | 0.614        | 0.993        | 1.405        |
| SimpleView [6]  | <b>93.9%</b>  | 1.047            | 0.872        | 0.715        | 1.242        | 1.357        | 0.983        | 0.844        | 1.316        |
| Point-DAE [39]  | 93.3%         | 0.835            | 0.957        | 0.883        | 0.944        | 0.841        | 0.668        | 0.815        | 0.736        |
| APCT [27]       | -             | 0.722            | 0.947        | 0.883        | 0.468        | 0.850        | 0.285        | <b>0.298</b> | 1.326        |
| Point-BERT [34] | 92.2%         | 1.098            | 0.872        | 1.503        | 0.512        | 0.903        | 1.085        | 1.495        | 1.316        |
| w. IDPT [36]    | 91.3%         | 1.142            | 0.936        | 0.813        | 0.512        | 1.174        | 2.071        | 1.353        | 1.135        |
| w. PSFT (Ours)  | 91.4%         | 0.790            | 1.149        | 0.503        | 0.383        | 0.961        | 0.319        | 0.625        | 1.591        |
| Point-MAE [15]  | 92.6%         | 0.983            | 0.840        | 1.256        | 0.464        | 0.763        | 1.075        | 1.265        | 1.219        |
| w. IDPT [36]    | 92.7%         | 1.008            | 0.851        | 0.769        | 0.613        | 1.000        | 1.346        | 1.145        | 1.330        |
| w. PSFT (Ours)  | 91.9%         | 0.704            | 1.032        | <b>0.427</b> | 0.431        | 0.860        | 0.298        | 0.502        | 1.377        |
| ULIP-2 [32]     | 92.1%         | 0.863            | 0.851        | 1.778        | 0.492        | <b>0.734</b> | 0.386        | 0.862        | 0.935        |
| w. IDPT [36]    | 92.9%         | 0.775            | <b>0.745</b> | 1.709        | 0.323        | 0.947        | 0.454        | 0.851        | 0.395        |
| w. PSFT (Ours)  | 93.3%         | <b>0.530</b>     | 0.819        | 0.585        | <b>0.298</b> | 0.816        | <b>0.244</b> | 0.567        | <b>0.381</b> |
| Uni3d-B [42]    | 92.9%         | 0.746            | 0.777        | 1.228        | 0.415        | 0.787        | 0.444        | 0.833        | 0.740        |
| w. IDPT [36]    | 91.5%         | 0.946            | 1.000        | 1.472        | 0.653        | 1.266        | 0.569        | 1.142        | 0.521        |
| w. PSFT (Ours)  | 93.4%         | 0.566            | 0.862        | 0.459        | 0.327        | 0.986        | 0.254        | 0.575        | 0.498        |

**Table 2. Error Rates on ModelNet40-C.** mER denotes mean error rate.

| Model           | mER $\downarrow$ | Density Corruptions |             |              |              |            | Noise Corruptions |             |            |             |             | Transformation Corruptions |            |             |             |             |
|-----------------|------------------|---------------------|-------------|--------------|--------------|------------|-------------------|-------------|------------|-------------|-------------|----------------------------|------------|-------------|-------------|-------------|
|                 |                  | Occlusion           | LiDAR       | Density Inc. | Density Dec. | Cutout     | Uniform           | Gaussian    | Impulse    | Up-sampling | Back-ground | Rotation                   | Shear      | FFD         | RBF         | Inv. RBF    |
| PointNet [16]   | 28.3             | 52.3                | 54.9        | 10.5         | 11.6         | 12.0       | 12.4              | 14.4        | 29.1       | 14.0        | 93.6        | 36.8                       | 25.4       | 21.3        | 18.6        | 17.8        |
| SimpleView [6]  | 27.2             | 55.5                | 82.2        | 13.7         | 17.2         | 20.1       | 14.5              | 14.2        | 24.6       | 17.7        | 46.8        | 30.7                       | 18.5       | 17.0        | 17.9        | 17.2        |
| RSCNN [13]      | 26.2             | 51.8                | 68.4        | 16.8         | 13.2         | 13.8       | 24.6              | 18.3        | 46.2       | 20.1        | 18.3        | 29.2                       | 17.0       | 18.1        | 19.2        | 18.6        |
| DGCNN [28]      | 25.9             | 59.2                | 81.0        | 14.1         | 17.3         | 15.4       | 14.6              | 16.6        | 24.9       | 19.1        | 53.1        | 19.1                       | 12.1       | 13.1        | 14.5        | 14.0        |
| PCT [7]         | 25.5             | 56.6                | 76.7        | 11.8         | 14.3         | 14.5       | 12.1              | 13.9        | 39.1       | 17.4        | 57.9        | 18.1                       | 11.5       | 12.4        | 13.0        | 12.6        |
| PointNet++ [17] | 23.6             | 54.7                | 66.5        | 16.0         | 10.0         | 10.7       | 20.4              | 16.4        | 35.1       | 17.2        | 18.6        | 27.6                       | 13.4       | 15.2        | 16.4        | 15.4        |
| Point-BERT [34] | 26.5             | 54.7                | 73.8        | 10.8         | 13.4         | 13.6       | 17.5              | 22.8        | 46.7       | 23.8        | 36.7        | 25.3                       | 13.3       | 14.5        | 15.9        | 14.9        |
| w. IDPT [36]    | 26.7             | 59.3                | 80.8        | 10.7         | 14.1         | 13.8       | 11.9              | 13.7        | 32.7       | 14.6        | 72.4        | 21.8                       | 12.9       | 14.3        | 14.1        | 12.8        |
| w. PSFT (Ours)  | 20.0             | 56.9                | 70.9        | 9.2          | 10.6         | 9.6        | 11.1              | 11.2        | 10.4       | 10.2        | 9.9         | 30.7                       | 16.9       | 15.5        | 14.1        | 13.5        |
| Point-MAE [15]  | 23.9             | 53.5                | 74.8        | 10.7         | 11.3         | 11.9       | 15.7              | 18.5        | 33.0       | 18.8        | 35.5        | 22.7                       | 11.5       | 13.1        | 13.9        | 13.0        |
| w. IDPT [36]    | 24.6             | 55.3                | 79.5        | 11.1         | 13.3         | 13.8       | 11.0              | 12.9        | 18.0       | 13.4        | 57.2        | 26.4                       | 13.5       | 15.0        | 14.7        | 13.8        |
| w. PSFT (Ours)  | 18.3             | 55.5                | 66.6        | 9.0          | 10.7         | 10.1       | 10.5              | <b>10.3</b> | 9.4        | 9.6         | 9.6         | 24.3                       | 12.9       | 12.9        | <b>12.0</b> | <b>11.3</b> |
| ULIP-2 [32]     | 24.1             | 51.4                | 80.0        | 9.7          | 10.8         | 10.9       | 29.9              | 35.7        | 18.7       | 31.9        | 12.9        | 18.6                       | 10.5       | 12.1        | 14.7        | 13.8        |
| w. IDPT [36]    | 23.3             | <b>42.6</b>         | 79.1        | 10.0         | 11.6         | 12.3       | 29.7              | 37.4        | 15.4       | 32.0        | 16.1        | 8.2                        | 7.8        | 12.8        | 17.6        | 16.4        |
| w. PSFT (Ours)  | 14.9             | 44.6                | 53.3        | <b>7.5</b>   | <b>8.8</b>   | <b>9.0</b> | 12.0              | 12.8        | <b>8.2</b> | <b>9.5</b>  | <b>7.6</b>  | <b>7.6</b>                 | <b>7.1</b> | <b>10.4</b> | 12.5        | 11.9        |
| Uni3d-B [42]    | 18.9             | 44.2                | 58.2        | 9.6          | 10.3         | 11.3       | 15.1              | 18.9        | 20.2       | 21.6        | 14.1        | 13.4                       | 8.8        | 11.7        | 13.4        | 12.6        |
| w. IDPT [36]    | 23.0             | <b>38.2</b>         | 56.3        | 10.5         | 11.8         | 16.7       | 22.0              | 29.3        | 26.4       | 33.2        | 17.3        | 10.0                       | 10.7       | 18.1        | 22.8        | 22.1        |
| w. PSFT (Ours)  | <b>14.2</b>      | 41.2                | <b>42.4</b> | 8.1          | 9.8          | 10.5       | <b>10.0</b>       | 10.8        | 8.9        | 9.7         | 7.6         | 9.0                        | 7.5        | 11.1        | 13.3        | 12.7        |

observe that each module contributes to robustness improvements over the full fine-tuning baseline. In particular, using only the point selection module already yields a clear gain on all three benchmarks, and adding either prompt generation or feature filtering further improves robustness. Combining all three modules gives the strongest overall performance on ModelNet-C and ModelNet40-C, reaching an mCE of 0.530 and an mER of 14.9, while on ScanObjectNN-C the PS+FFM variant achieves the lowest mCE, indicating a slightly different trade-off on real-world corruptions.

*Effect of Test-Time  $\alpha$  under Different Training-Time  $\lambda$  Settings.* Figure 4 illustrates how the test-time blending parameter  $\alpha$  affects robustness (mCE) under three training-time Beta distributions: Beta(0.5, 0.5), Beta(1.0, 1.0), and Beta(1.5, 1.5). For clarity, the key observations are:

**Table 3. Experimental Results on ScanObjectNN-C.** OA and mCE denote overall accuracy and mean corruption error, respectively.

| Model           | OA $\uparrow$ | mCE $\downarrow$ | Scale        | Jitter       | Drop-G       | Drop-L       | Add-G        | Add-L        | Rotate       |
|-----------------|---------------|------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| RPC [21]        | 74.7%         | 1.326            | 1.317        | 1.073        | 1.455        | 1.305        | 1.142        | 1.587        | 1.402        |
| PointNet++ [17] | 86.2%         | 0.969            | 0.897        | 1.103        | 0.550        | 1.277        | 0.947        | 0.905        | 1.107        |
| PointNeXt [19]  | 87.3%         | 0.921            | 0.803        | 1.079        | 0.807        | 0.942        | 0.944        | 0.875        | 0.995        |
| PointGPT [3]    | -             | 0.847            | 1.199        | 1.040        | 0.585        | 0.769        | 0.361        | 0.722        | 1.251        |
| APCT [27]       | -             | 0.742            | 1.045        | 0.989        | 0.489        | 0.670        | <b>0.300</b> | <b>0.630</b> | 1.071        |
| Point-BERT [34] | 83.4%         | 0.931            | 0.618        | 1.088        | 0.587        | 0.736        | 0.661        | 1.630        | 1.195        |
| w. IDPT [36]    | 83.6%         | 1.053            | 0.749        | 0.958        | 0.545        | 0.851        | 1.117        | 2.154        | 0.996        |
| w. PSFT (Ours)  | 79.4%         | 0.970            | 0.898        | 0.954        | 0.608        | 0.937        | 0.493        | 1.612        | 1.288        |
| Point-MAE [15]  | 84.3%         | 0.895            | 0.609        | 0.952        | 0.569        | 0.726        | 0.550        | 1.722        | 1.135        |
| w. IDPT [36]    | 85.9%         | 0.982            | 0.585        | 0.910        | 0.511        | 0.733        | 1.126        | 2.044        | 0.963        |
| w. PSFT (Ours)  | 77.6%         | 1.014            | 0.960        | 0.974        | 0.701        | 0.990        | 0.580        | 1.568        | 1.326        |
| ULIP-2 [32]     | 86.2%         | 0.777            | 0.538        | 1.096        | 0.524        | <b>0.637</b> | 0.376        | 1.348        | 0.918        |
| w. IDPT [36]    | <b>87.3%</b>  | 0.792            | 0.533        | 1.031        | <b>0.399</b> | 0.673        | 0.654        | 1.670        | <b>0.584</b> |
| w. PSFT (Ours)  | 86.1%         | <b>0.685</b>     | 0.621        | 0.960        | 0.431        | 0.670        | 0.333        | 1.079        | 0.700        |
| Uni3d-B [42]    | 85.8%         | 0.801            | <b>0.509</b> | 1.085        | 0.532        | 0.680        | 0.365        | 1.502        | 0.933        |
| w. IDPT [36]    | 83.6%         | 0.946            | 0.713        | 1.138        | 0.696        | 0.967        | 0.491        | 1.771        | 0.846        |
| w. PSFT (Ours)  | 86.4%         | 0.702            | 0.626        | <b>0.871</b> | 0.458        | 0.733        | 0.341        | 1.079        | 0.805        |

(i) **Optimal test-time  $\alpha$  is near 0.5 across all settings:** mCE is minimized when  $\alpha \approx 0.5$ , indicating that a balanced residual mixture of the identity and transformed features ( $x$  and  $f(x)$ ) yields the strongest robustness.

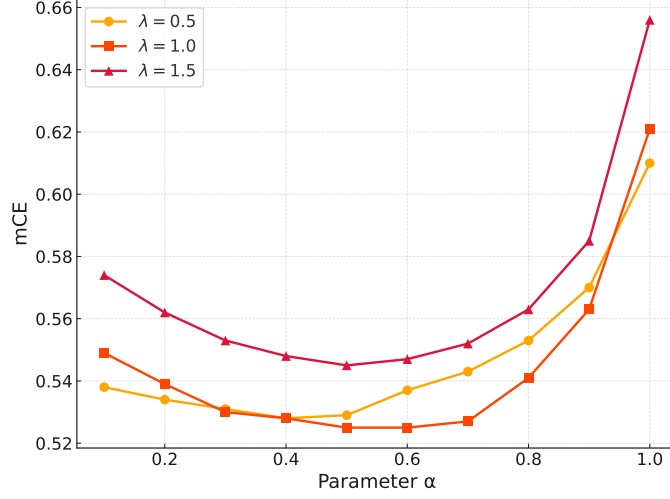
(ii) **Beta(0.5, 0.5) gives the strongest overall robustness profile:** assigning more probability mass near boundaries exposes training to broader residual-mixing patterns and consistently lowers mCE.

(iii) **Robustness degrades as  $\alpha \rightarrow 1.0$ :** across all training configurations, rapidly increasing mCE suggests that excessive reliance on the identity path weakens the corrective effect of  $f(x)$  during inference.

(iv) **Larger  $\lambda$  narrows residual-mixing diversity:** compared with  $\lambda = 0.5$ , Beta(1.0, 1.0) and Beta(1.5, 1.5) correspond to weaker robustness, indicating that reducing extreme residual weights is less effective than broader mixing exposure.

Overall, stochastic residual blending with broad training-time coverage is beneficial, and test-time  $\alpha \approx 0.5$  provides a stable trade-off between denoising and information preservation.

*Comparison with Other Fine-Tuning Strategies.* We conduct experiments on ModelNet-C [21] using ULIP-2 to compare PSFT with representative PEFT baselines, including VPT [8] and AdaptFormer [4]. As shown in Table 4, AdaptFormer reduces the number of tunable parameters but still trails full fine-tuning in robustness, while VPT performs substantially worse under corruption. In contrast, PSFT improves robustness while preserving parameter efficiency. Compared to full fine-tuning, PSFT reduces the number of trainable parameters from 22.1M to 2.4M and lowers mCE from 0.863 to 0.530.



**Fig. 4. Effect of Test-Time Blending Coefficient  $\alpha$ .** The plot shows mCE versus test-time  $\alpha$  under training-time sampling distributions  $\text{Beta}(\lambda, \lambda)$  with  $\lambda \in \{0.5, 1.0, 1.5\}$ . Across all settings, robustness is best around  $\alpha \approx 0.5$ , and training with  $\text{Beta}(0.5, 0.5)$  yields the lowest overall mCE.

**Table 4. Comparison with Fine-Tuning Baselines on ULIP-2 (ModelNet-C).** #TP denotes the number of trainable parameters.

| Method                | #TP (M) | mCE          |
|-----------------------|---------|--------------|
| w. FT                 | 22.1    | 0.863        |
| w. VPT-Deep           | 0.4     | 1.454        |
| w. AdaptFormer        | 0.9     | 0.920        |
| w. <b>PSFT (ours)</b> | 2.4     | <b>0.530</b> |

**Table 5. Ablation Study on ModelNet-C [21], ModelNet40-C [23], and ScanObjectNN-C [25].** PS, PG, and FFM denote the point selection, prompt generation, and feature filter modules, respectively.

| Components   |              |              | ModelNet-C [21] |                  | ModelNet40-C [23] | ScanObjectNN-C [25] |                  |
|--------------|--------------|--------------|-----------------|------------------|-------------------|---------------------|------------------|
| PS           | PG           | FFM          | OA $\uparrow$   | mCE $\downarrow$ | mER $\downarrow$  | OA $\uparrow$       | mCE $\downarrow$ |
| $\times$     | $\times$     | $\times$     | 92.1%           | 0.863            | 24.1              | 86.2%               | 0.777            |
| $\times$     | $\times$     | $\checkmark$ | 93.6%           | 0.819            | 24.6              | 87.4%               | 0.735            |
| $\times$     | $\checkmark$ | $\times$     | 92.6%           | 0.821            | 20.3              | 85.3%               | 0.817            |
| $\checkmark$ | $\times$     | $\times$     | 91.9%           | 0.656            | 17.6              | 85.4%               | 0.724            |
| $\times$     | $\checkmark$ | $\checkmark$ | 93.4%           | 0.734            | 19.9              | 86.9%               | 0.758            |
| $\checkmark$ | $\times$     | $\checkmark$ | <b>94.1%</b>    | 0.558            | 15.8              | <b>87.4%</b>        | <b>0.654</b>     |
| $\checkmark$ | $\checkmark$ | $\times$     | 92.4%           | 0.648            | 16.8              | 84.5%               | 0.780            |
| $\checkmark$ | $\checkmark$ | $\checkmark$ | 93.3%           | <b>0.530</b>     | <b>14.9</b>       | 86.1%               | 0.685            |

Thus, PSFT uses 10.9% of the trainable parameters of full fine-tuning, corresponding to an 89.1% reduction.

*Effect of Point Selection on Enhancing Robustness* Without point selection, noisy points directly enter the prompt generator and degrade prompt quality.

Our ablation shows that prompt generation already improves robustness over full fine-tuning, and combining it with point selection brings further gains. This verifies that selecting minimally influential points effectively shields the PEFT branch from corrupted inputs.

*Robust and Efficient Task-Oriented Prompt Generation.* Our prompt generation module adapts the frozen pre-trained model by producing task-conditioned prompt tokens from selected points and injecting them into transformer layers before global pooling. Because the prompt branch starts from filtered points rather than raw corrupted inputs, it operates on a cleaner representation with reduced noise propagation. This enables task adaptation while preserving the robustness of the frozen backbone.

## 5 Conclusion

In this paper, we present PSFT, a point-selection fine-tuning framework for robust adaptation of 3D pre-trained models. PSFT combines minimally influential point selection, layer-wise prompt tuning on a frozen backbone, and a lightweight feature filter with stochastic residual blending. Experiments show consistent robustness gains on ModelNet-C and ModelNet40-C, together with improvements for ULIP-2 and Uni3D-B on ScanObjectNN-C, while keeping trainable parameters low. In future work, we will evaluate PSFT under more complex and realistic 3D corruption settings to better characterize robustness and generalization in practical scenarios.

**Acknowledgements** This work was supported by the Shenzhen Science and Technology Program under Grant KJZD20240903104400001 and was financially supported by the Research Task Assignment Project from the Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), under Grant No. GML-26420004.

## References

1. Ai, Z., Liu, Z., Lei, Y., Cui, Z., Zou, X., Zhou, J.: Gaprompt: Geometry-aware point cloud prompt for 3d vision model. In: ICML. pp. 796–808 (2025)
2. Cai, H., Yin, D., Yu, F.R., Xiong, S.: DSTR: Dual scenes transformer for cross-modal fusion in 3d object detection. In: WACV. pp. 3064–3073 (2025). <https://doi.org/10.1109/WACV61041.2025.00303>
3. Chen, G., Wang, M., Yang, Y., Yu, K., Yuan, L., Yue, Y.: Pointgpt: Auto-regressively generative pre-training from point clouds. In: NeurIPS. vol. 36, pp. 29667–29679 (2023)
4. Chen, S., Ge, C., Tong, Z., Wang, J., Song, Y., Wang, J., Luo, P.: Adaptformer: Adapting vision transformers for scalable visual recognition. In: NeurIPS. vol. 35, pp. 16664–16678 (2022)
5. Choy, C.B., Gwak, J., Savarese, S.: 4d spatio-temporal convnets: Minkowski convolutional neural networks. In: CVPR. pp. 3075–3084 (2019)

6. Goyal, A., Law, H., Liu, B., Newell, A., Deng, J.: Revisiting point cloud shape classification with a simple and effective baseline. In: ICML. vol. 139, pp. 3809–3820 (2021)
7. Guo, M.H., Cai, J.X., Liu, Z.N., Mu, T.J., Martin, R.R., Hu, S.M.: Pct: Point cloud transformer. *Computational Visual Media* **7**, 187–199 (2021). <https://doi.org/10.1007/s41095-021-0229-5>
8. Jia, M., Tang, L., Chen, B.C., Cardie, C., Belongie, S., Hariharan, B., Lim, S.N.: Visual prompt tuning. In: ECCV. pp. 709–727 (2022)
9. Levi, M.Y., Gilboa, G.: Epic: Ensemble of partial point clouds for robust classification. In: ICCV. pp. 14475–14484 (2023)
10. Levi, M.Y., Gilboa, G.: Robustifying point cloud networks by refocusing. In: 3DV. pp. 1144–1159 (2025). <https://doi.org/10.1109/3DV66043.2025.00110>
11. Li, K., Ferrand, J.C.N., Sheatsley, R., Hoak, B., Beugin, Y., Pauley, E., McDaniel, P.: On the robustness tradeoff in fine-tuning. In: ICCV. pp. 4898–4907 (2025)
12. Liu, H., Jia, J., Gong, N.Z.: Pointguard: Provably robust 3d point cloud classification. In: CVPR. pp. 6186–6195 (2021)
13. Liu, Y., Fan, B., Xiang, S., Pan, C.: Relation-shape convolutional neural network for point cloud analysis. In: CVPR. pp. 8894–8903 (2019). <https://doi.org/10.1109/CVPR.2019.00910>
14. Ma, X., Qin, C., You, H., Ran, H., Fu, Y.: Rethinking network design and local geometry in point cloud: A simple residual MLP framework. In: ICLR (2022)
15. Pang, Y., Wang, W., Tay, F.E., Liu, W., Tian, Y., Yuan, L.: Masked autoencoders for point cloud self-supervised learning. In: ECCV. pp. 604–621 (2022)
16. Qi, C.R., Su, H., Mo, K., Guibas, L.J.: Pointnet: Deep learning on point sets for 3d classification and segmentation. In: CVPR. pp. 77–85 (2017)
17. Qi, C.R., Yi, L., Su, H., Guibas, L.J.: Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In: NeurIPS. pp. 5099–5108 (2017)
18. Qi, Z., Dong, R., Fan, G., Ge, Z., Zhang, X., Ma, K., Yi, L.: Contrast with reconstruct: Contrastive 3d representation learning guided by generative pretraining. In: ICML. vol. 202, pp. 28223–28243 (2023)
19. Qian, G., Li, Y., Peng, H., Mai, J., Hammoud, H., Elhoseiny, M., Ghanem, B.: Pointnext: Revisiting pointnet++ with improved training and scaling strategies. In: NeurIPS. vol. 35, pp. 23192–23204 (2022)
20. Rakotosaona, M.J., La Barbera, V., Guerrero, P., Mitra, N.J., Ovsjanikov, M.: Pointcleannet: Learning to denoise and remove outliers from dense point clouds. *Comput. Graph. Forum* **39**(1), 185–203 (2020). <https://doi.org/10.1111/cgf.13753>
21. Ren, J., Pan, L., Liu, Z.: Benchmarking and analyzing point cloud classification under corruptions. In: ICML. vol. 162, pp. 18559–18575 (2022)
22. Sun, H., Wang, Y., Chen, W., Deng, H., Li, D.: Parameter-efficient prompt learning for 3d point cloud understanding. In: ICRA. pp. 9478–9486 (2024). <https://doi.org/10.1109/ICRA57147.2024.10610093>
23. Sun, J., Zhang, Q., Kailkhura, B., Yu, Z., Xiao, C., Mao, Z.M.: Benchmarking robustness of 3d point cloud recognition against common corruptions. *arXiv preprint arXiv:2201.12296* (2022)
24. Tang, Y., Zhang, R., Guo, Z., Ma, X., Zhao, B., Wang, Z., Wang, D., Li, X.: Pointpeft: Parameter-efficient fine-tuning for 3d pre-trained models. In: AAAI. vol. 38, pp. 5171–5179 (2024). <https://doi.org/10.1609/aaai.v38i6.28323>
25. Uy, M.A., Pham, Q., Hua, B., Nguyen, D.T., Yeung, S.: Revisiting point cloud classification: A new benchmark dataset and classification model on real-world data. In: ICCV. pp. 1588–1597 (2019)

26. Wang, J., Ding, L., Xu, T., Dong, S., Xu, X., Bai, L., Li, J.: Sample-adaptive augmentation for point cloud recognition against real-world corruptions. In: ICCV. pp. 14330–14339 (2023)
27. Wang, J., Xu, T., Ding, L., Li, J.: Target-guided adversarial point cloud transformer towards recognition against real-world corruptions. In: NeurIPS (2024)
28. Wang, Y., Sun, Y., Liu, Z., Sarma, S.E., Bronstein, M.M., Solomon, J.M.: Dynamic graph CNN for learning on point clouds. *ACM Trans. Graph.* **38**(5), 146:1–146:12 (2019). <https://doi.org/10.1145/3326362>
29. Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., Xiao, J.: 3d shapenets: A deep representation for volumetric shapes. In: CVPR. pp. 1912–1920 (2015)
30. Xu, M., Ding, R., Zhao, H., Qi, X.: PConv: position adaptive convolution with dynamic kernel assembling on point clouds. In: CVPR. pp. 3173–3182 (2021)
31. Xue, L., Gao, M., Xing, C., Martín-Martín, R., Wu, J., Xiong, C., Xu, R., Niebles, J.C., Savarese, S.: ULIP: learning a unified representation of language, images, and point clouds for 3d understanding. In: CVPR. pp. 1179–1189 (2023)
32. Xue, L., Yu, N., Zhang, S., Panagopoulou, A., Li, J., Martín-Martín, R., Wu, J., Xiong, C., Xu, R., Niebles, J.C., Savarese, S.: ULIP-2: towards scalable multimodal pre-training for 3d understanding. In: CVPR. pp. 27081–27091 (2024)
33. Yan, X., Zheng, C., Li, Z., Wang, S., Cui, S.: Pointasnl: Robust point clouds processing using nonlocal neural networks with adaptive sampling. In: CVPR. pp. 5589–5598 (2020)
34. Yu, X., Tang, L., Rao, Y., Huang, T., Zhou, J., Lu, J.: Point-bert: Pre-training 3d point cloud transformers with masked point modeling. In: CVPR. pp. 19313–19322 (2022)
35. Yuan, W., Held, D., Mertz, C., Hebert, M.: Iterative transformer network for 3d point cloud. *arXiv preprint arXiv:1811.11209* (2018)
36. Zha, Y., Wang, J., Dai, T., Chen, B., Wang, Z., Xia, S.T.: Instance-aware dynamic prompt tuning for pre-trained point cloud models. In: ICCV. pp. 14161–14170 (2023)
37. Zha, Y., Wang, C., Yang, W., Zhang, T., Wu, F.: Exploring vision semantic prompt for efficient point cloud understanding. In: ICML. pp. 74271–74287 (2025)
38. Zhang, J., Chen, W., Wang, Y., Vasudevan, R., Johnson-Roberson, M.: Point set voting for partial point cloud analysis. *IEEE Robotics Autom. Lett.* **6**(2), 596–603 (2021). <https://doi.org/10.1109/LRA.2020.3048658>
39. Zhang, Y., Lin, J., Li, R., Jia, K., Zhang, L.: Point-dae: Denoising autoencoders for self-supervised point cloud learning. *IEEE Trans. Neural Netw. Learn. Syst.* **36**(9), 16279–16293 (2025). <https://doi.org/10.1109/TNNLS.2025.3557055>
40. Zhao, C., Yang, J., Xiong, X., Zhu, A., Cao, Z., Li, X.: Rotation invariant point cloud analysis: Where local geometry meets global topology. *Pattern Recognition* **127**, 108626 (2022). <https://doi.org/10.1016/j.patcog.2022.108626>
41. Zhao, H., Jiang, L., Jia, J., Torr, P.H., Koltun, V.: Point transformer. In: CVPR. pp. 16259–16268 (2021)
42. Zhou, J., Wang, J., Ma, B., Liu, Y., Huang, T., Wang, X.: Uni3d: Exploring unified 3d representation at scale. In: ICLR (2024)
43. Zhou, X., Liang, D., Xu, W., Zhu, X., Xu, Y., Zou, Z., Bai, X.: Dynamic adapter meets prompt tuning: Parameter-efficient transfer learning for point cloud analysis. In: CVPR. pp. 14707–14717 (2024)